



Operational Decisions in AGV-Served Flowshop Loops: Fleet Sizing and Decomposition

NICHOLAS G. HALL
The Ohio State University, USA

CHELLIAH SRISKANDARAJAH
The University of Texas at Dallas, USA

THARMARAJAH GANESHAARAJAH
ADGA Consulting, Ottawa, Canada

Abstract. This paper considers design and operational issues that arise in repetitive manufacturing systems served by automated guided vehicles (AGVs) in loop layouts with unidirectional material flow. Such systems are in widespread industrial use, and play an important role in modern manufacturing environments. The objective considered is the minimization of AGV fleet size, given the minimum steady state cycle time required to produce a minimal job set (or equivalently, given the maximum throughput rate). We also study whether the decomposition of a large AGV-served flowshop loop into several smaller loops improves productivity. The original loop and the decomposed design are compared with respect to the minimum cycle time needed for the repetitive manufacture of a minimal job set. When there are three or more machines in the loop, finding the optimal cycle time is an intractable problem. We therefore use the genetic algorithm developed in the companion paper [17], to identify whether the original or the decomposed design is more efficient. Our study suggests that many systems perform more productively as a result of decomposition. Finally, we discuss a joint sequencing issue that arises in decomposed systems with limited buffers between the loops, and analyze the tractability of all the relevant joint sequencing problems.

Keywords: manufacturing, automated guided vehicles, fleet sizing, decomposition

1. Introduction

Following their first industrial use in 1974, the popularity of automated guided vehicles (AGVs) grew rapidly, so that more than 15,000 AGVs were in use within little more than a decade thereafter [36]. AGVs are now an important part of many modern manufacturing systems. Their advantages, and the motivation for using loop layouts, are discussed in the companion paper by Hall, Sriskandarajah and Ganesharajah [17]. The next three paragraphs describe and motivate the three objectives of this paper.

Single loop layouts represent an attractive compromise for the design of AGV layouts. Within such loops, several machines are arranged in a sequence, as in a flowshop [1,32]. Jobs are moved on pallets between machines by AGVs. The material flow pattern is unidirectional within the loop. AGV-served loop layouts are widely used in industry. Gould [11] describes an interesting application at the Kansas City plant of Ford Motor

Company, where 18 AGVs transport instrument panels through a series of 22 workstations arranged in a single 600ft loop. Since AGVs are expensive, determining an optimal fleet size is an important issue. Therefore, the first objective of this paper is to develop and test procedures for solving that problem.

In an AGV-served manufacturing system, an AGV may be required to visit any machine in the system. As a result, AGV conflict may occur. In order to reduce this conflict, Bozer and Srinivasan [4] suggest a *tandem AGV configuration*, in which the entire manufacturing system is divided into many nonoverlapping loops, each served by a single AGV. Material transfer between loops is achieved by providing buffers between loops. Since no two AGVs meet in the tandem configuration, the time lost due to AGV conflict is totally eliminated. However, additional time may be needed for a job to wait in the interloop buffer before it is picked up by another AGV. Due to this tradeoff, the alternative of tandem configuration requires careful evaluation. Related research issues include how to implement the decomposition in the facility layout. For example, there is a partitioning problem that groups the machines so that one AGV serves each group in a single loop. Also, there is a load routing problem to select a route for transferring a load between any two loops. Therefore, the second objective of this paper is to provide a detailed analysis of the advantages and disadvantages of tandem configurations relative to conventional ones.

As a result of a decision to decompose a large loop layout, it is necessary to maintain storage buffers between the loops. Work-in-process may accumulate expensively in these buffers, due partly to a need for resequencing and partly to timing mismatches between the loops. In order to eliminate the first of these problems, we consider the joint sequencing of the loops. Therefore, the third objective of this paper is to map the computational complexity of the joint sequencing problem across various layout configurations.

In line with the just-in-time philosophy [18,29] of modern manufacturing, we consider the minimization of the *steady state* cycle time required to produce a *minimal job set* (MJS). An MJS is the smallest possible set of jobs having the same proportions as the day's production target [35]. For example, if the total day's production target is 300 units of type A, 500 units of type B, and 900 units of type C, the MJS consists of 17 jobs: three of A, five of B, and nine of C. A steady state is a condition within an AGV system in which the waiting time of the AGV for each job at each machine, and the parking time of the AGV at the input station before dispatch with each job, are the same for each MJS. Steady state cycle time minimization and long run throughput rate maximization are equivalent.

We propose both analytical and computational methods for comparing the steady state cycle times of the single and decomposed AGV-served systems. To estimate the cycle time of an AGV-served flowshop loop, we use results from the companion paper by Hall, Sriskandarajah and Ganesharajah [17], in which the computational complexity of several scheduling problems is determined. We make use of all the polynomial time algorithms there, and propose heuristics for the intractable problems. These heuristics perform well, relative to lower bounds, in computational testing.

This paper is organized as follows. In section 2, we briefly review the literature related to AGV-served manufacturing systems, and in particular to their decomposition into smaller systems. Section 3 lists the notation used in this paper and provides a detailed description of the flowshop loop and the AGV dispatching policy studied. Section 4 proposes an approach for finding an optimal, or near optimal, fleet size for AGV loops. In section 5, we investigate whether loop decomposition is advantageous. The tractability of finding an optimal common job sequence for all the decomposed loops is discussed in section 6. Section 7 contains a conclusion and some suggestions for future research.

2. Literature review

The design and operational issues that arise in AGV-served manufacturing systems are discussed in detail by Ganesharajah, Hall and Sriskandarajah [8], therefore we provide only a brief summary here. The study of Maxwell and Muckstadt [28] on the fleet sizing issue initiated research on AGV systems. Grosseschallau and Kusiak [14] describe an expert system for the fleet sizing problem with multiple objectives. Stern and Vitter [33], Kise, Shioyama and Ibaraki [21] and Panwalkar [31] introduce AGV material handling issues into classical scheduling models. Blazewicz et al. [2,3] study the simultaneous scheduling and routing of jobs among identical parallel machines organized in a loop layout. Hall, Sriskandarajah and Ganesharajah [17] study the effectiveness of three widely used AGV dispatching policies for the objective of minimizing the cycle time to produce an MJS repetitively.

Bozer and Srinivasan [5,6] discuss the advantages and disadvantages of tandem configurations. The advantages include the complete elimination of AGV conflict, easier controllability, the effective use of bidirectional AGVs, and easier expansion by adding new loops. The disadvantages include additional time spent loading and unloading, less tolerance to breakdowns, and additional transit time incurred by loads that wait at inter-loop buffers. They use simulation based on a set partitioning heuristic to compare the throughput rates of conventional and tandem AGV systems. Their conclusions are that tandem configurations perform better for manufacturing systems with a large number of AGVs, but conventional systems work better with few AGVs. They also call for research on the partitioning and load routing problems.

Lin, Chang and Liu [26] present a two phase approach for the load routing problem. Lin and Dgen [27] propose a graph theoretic procedure in which each free time window of an AGV is represented as a node in a graph. Wang and Hafeez [34] compare the performance of conventional and tandem AGV configurations using generalized stochastic Petri nets. In their computational study for a three machine FMS, tandem AGV systems provide a higher throughput rate than conventional ones. Choi, Kwon and Lee [7] perform a similar comparison using simulation, and conclude that tandem systems have a higher throughput rate than conventional systems when jobs arrive according to a Poisson process.

3. Preliminaries

3.1. Notation

The following notation is used to describe an AGV system.

$M_1, \dots, M_m$: the machines in the system.

v : the number of AGVs in the system (the fleet size).

I : the input station (where jobs enter the system).

O : the output station (where jobs leave the system).

MJS: the minimal job set consisting of r_j jobs of type P_j .

k : the number of part-types in the MJS.

$n = r_1 + \dots + r_k$: the total number of jobs to be produced in the MJS.

a_i (respectively, b_i): the processing time of job i on machine M_1 (respectively, M_2).

$p_{i,q}$: the processing time of job i on machine M_q . This notation is used when $m > 2$.

l : the number of invisible *zones* in the loop.

δ : the time taken by an AGV to traverse a single zone.

ε : the time taken by an AGV to pick up or drop off a job.

$\gamma = l\delta + 2(m+1)\varepsilon$: the total activity time (turnaround time) of an AGV in the system.

$x = l_1\delta + 2\varepsilon$: the time to pick up a job from I , move it to M_1 , and load it onto M_1 .

$y = l_2\delta + 2\varepsilon$: the time to pick up a finished job from M_1 , move it to M_2 , and load it onto M_2 .

$z = l_3\delta + l_4\delta + 2\varepsilon$: in a two machine loop, the time to pick up a job from M_2 , move it to O , drop it at O , and return to I .

$\theta = \delta + 2\varepsilon$: the minimum time gap between consecutive AGVs to avoid AGV conflict under a No Wait AGV operating policy.

$\sigma(j)$: the j th job in the cyclic sequence σ .

$T_{\sigma(i)\sigma(i+1)}$: the time between the pick ups of job $J_{\sigma(i)}$ and job $J_{\sigma(i+1)}$ at the input station I .

T_σ : the cycle time to produce an MJS in the sequence σ , where $T_\sigma = T_{\sigma(1)\sigma(2)} + T_{\sigma(2)\sigma(3)} + \dots + T_{\sigma(n)\sigma(1)}$.

w_q^i : the waiting time of an AGV in front of machine M_q to pick up job $J_{\sigma(i)}$.

The standard classification scheme for scheduling problems [12] can be denoted $\psi_1|\psi_2|\psi_3$, where ψ_1 indicates the scheduling environment, ψ_2 describes the job characteristics or restrictive requirements, and ψ_3 defines the objective function to be minimized. This scheme is extended here to provide for the classification of scheduling problems arising in flowshop loop layouts served by AGVs.

Under ψ_1 , we have

AGV_v, lpm : an AGV-served system with v vehicles and a loop layout with m machines.

Under ψ_2 , we may have

$k \geq 2$: jobs may be of different types.

nw : a No Wait policy is used for AGV dispatching.

$C_t = C_t^*$: the steady state cycle time required to produce an MJS is optimal.

Under ψ_3 , we may wish to minimize

C_t : the steady state cycle time required to produce an MJS.

v : the number of vehicles used.

For example, problem $AGV, lp2|k \geq 2, nw, C_t = C_t^*|v$ denotes fleet size minimization in a loop layout with two machines, where the MJS consists of more than one job type, where AGVs are dispatched under the No Wait policy, and where the minimum possible cycle time for any fleet size is achieved.

3.2. System and dispatching policies

In the two machine loop shown in figure 1, the AGV picks up a job at the input point I , moves it to machine M_1 , then loads it onto M_1 , waits until processing is over, then unloads the job from M_1 , moves it to M_2 and so on, until finally it brings the finished job to the output point O . After dropping off the job at O , it goes to I to pick up the next job. An AGV can be operated under one of three dispatching policies [13]. The one we consider is now described.

No Wait (NW) Policy: Under this policy, an AGV is dispatched from I so that it does not wait elsewhere in the loop. That is, a job is always completed by the time an AGV arrives at any machine. Each AGV is delayed at I for the minimum time necessary to achieve this.

Hall, Sriskandarajah and Ganesharajah [17] study the NW policy, which is attractive from a practical point of view because of its conceptual simplicity and its ease of implementation. Table 1 summarizes complexity results for scheduling problems under the NW policy, as proved by Hall, Sriskandarajah and Ganesharajah [17] (denoted here by HSG [17]). See Garey and Johnson [9] for related definitions.

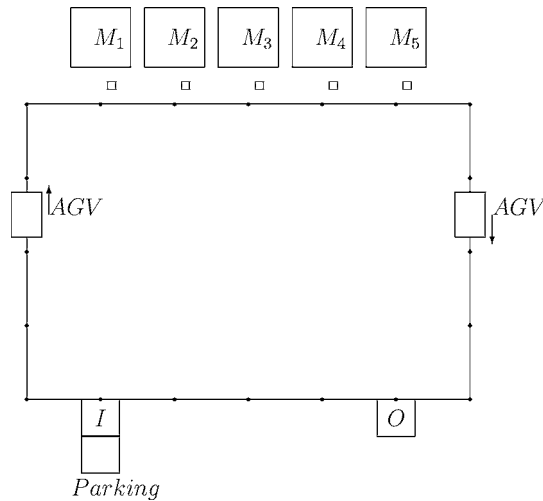


Figure 1. A large AGV-served flowshop loop.

Table 1
Complexity of scheduling under NW policy.

Problem	Complexity	Comments
$AGV1, lp2 k \geq 2, nw C_t$	$O(n \log n)$	HSG [17]
$AGV2, lp2 k \geq 2, nw C_t$	Open	
$AGV3, lp2 k \geq 2, nw C_t$	Unary NP-complete	HSG [17]

4. Fleet sizing

We begin with a discussion of scheduling issues that are closely related to fleet sizing. The Gantt chart for a schedule of a two machine loop under the NW policy is shown in figure 2.

4.1. Scheduling

Hall, Sriskandarajah and Ganesharajah [17] establish the following result.

Lemma 1. In problem $AGV1, lp2|k \geq 2, nw|C_t$, the cycle time of an MJS in a sequence σ is

$$T_\sigma = \sum_{i=1}^n \max\{a_{\sigma(i+1)}, b_{\sigma(i)}, \gamma\},$$

where $\gamma = l\delta + 6\varepsilon$ and we define $a_{\sigma(n+1)} = a_{\sigma(1)}$.

Proof. In a steady state under the NW policy, assume that job $J_{\sigma(i)}$ is being processed at M_1 and job $J_{\sigma(i-1)}$ is being processed at M_2 . The activities of an AGV in a cycle and their times follow: pick up job $J_{\sigma(i+1)}$ at I : (ε), move to M_1 : ($l_1\delta$), load job $J_{\sigma(i+1)}$ at the input buffer at M_1 : (ε), unload job $J_{\sigma(i)}$ from the output buffer at M_1 : (ε), move to M_2 : ($l_2\delta$), load job $J_{\sigma(i)}$ at the input buffer at M_2 : (ε), unload job $J_{\sigma(i-1)}$ from the output buffer at M_2 : (ε), move to O : ($l_3\delta$), drop job $J_{\sigma(i-1)}$ off at O : (ε), move to I : ($l_4\delta$), wait at I before loading job $J_{\sigma(i+2)}$ on M_1 : (d_{i+1}).

Note that d_{i+1} is the delay time of an AGV at I before it picks up job $J_{\sigma(i+2)}$ at I and reenters the loop. Since $l = l_1 + l_2 + l_3 + l_4$, the time between the pick up of job $J_{\sigma(i+1)}$ and job $J_{\sigma(i+2)}$ at I is given by $T_{\sigma(i+1)\sigma(i+2)} = l\delta + 6\varepsilon + d_{i+1}$. Now, the delay time, d_{i+1} , must be larger than or equal to $\max\{0, a_{\sigma(i+1)} - l\delta - 6\varepsilon\}$, since when the AGV reaches M_1 , job $J_{\sigma(i+1)}$ must be completed. Similarly, d_{i+1} must also be larger than or equal to $\max\{0, b_{\sigma(i)} - l\delta - 6\varepsilon\}$, since when the AGV reaches M_2 , job $J_{\sigma(i)}$ must be completed. Therefore, the minimum delay time is $d_{i+1} = \max\{0, a_{\sigma(i+1)} - \gamma, b_{\sigma(i)} - \gamma\}$, where $\gamma = l\delta + 6\varepsilon$. By substituting for d_{i+1} , we obtain $T_{\sigma(i+1)\sigma(i+2)} = \max\{\gamma, a_{\sigma(i+1)}, b_{\sigma(i)}\}$. Thus, the expression for cycle time $T_\sigma = \sum_{i=1}^n T_{\sigma(i)\sigma(i+1)}$ can be derived as shown. $\square$

Consider the schedule shown in figure 2. Here we define

$$h_i = \max\{0, \gamma - \max\{a_{\sigma(i)}, b_{\sigma(i-1)}\}\},$$

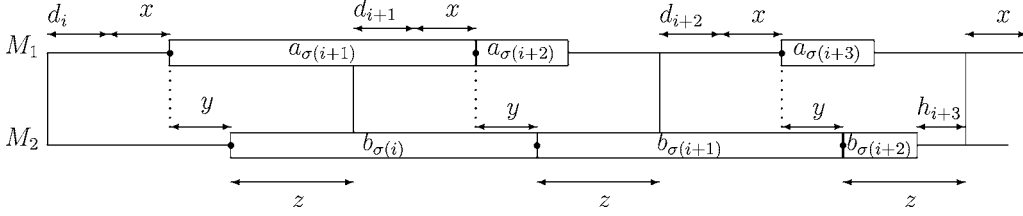


Figure 2. Gantt chart for a two machine loop under the NW policy.

which is the idle time that occurs between the start of jobs $J_{\sigma(i)}$ and $J_{\sigma(i+1)}$ on M_1 . If $d_i = 0$ then $h_i \geq 0$, otherwise $h_i = 0$.

In lemma 1, the cycle time formula of an MJS in a sequence σ for problem $AGV1, lp2|k \geq 2, nw|C_t$ is given by

$$T_\sigma = \sum_{i=1}^n \max\{a_{\sigma(i+1)}, b_{\sigma(i)}, \gamma\},$$

where $\gamma = l\delta + 6\varepsilon$ and $a_{\sigma(n+1)} = a_{\sigma(1)}$. This expression can be written as

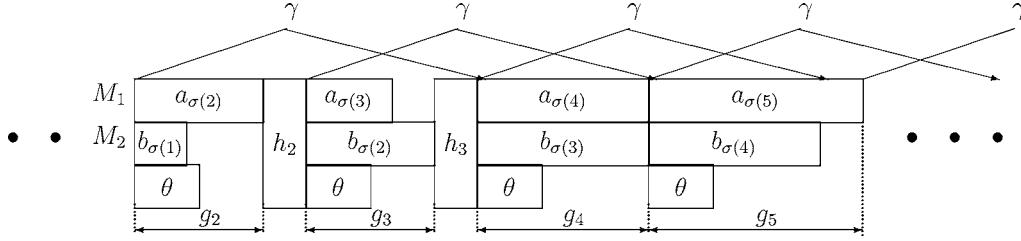
$$T_\sigma = \sum_{i=1}^n \max\{e_{\sigma(i+1)}, f_{\sigma(i)}\},$$

where $e_{\sigma(i+1)} = a_{\sigma(i+1)}$, and $f_{\sigma(i)} = \max\{b_{\sigma(i)}, \gamma\}$. Hence, the job sequence that minimizes the cycle time can be found using the algorithm of Gilmore and Gomory [10], which can be implemented in $O(n \log n)$ time.

Note that the cycle time expression shown in lemma 1 for a two machine loop with one AGV can be generalized to loops with $m \geq 2$ machines and one AGV. In problem $AGV1, lpm|k \geq 2, nw|C_t$, the cycle time of an MJS in a sequence σ is

$$T_\sigma = \sum_{i=1}^n \max\{p_{\sigma(i+m-1),1}, p_{\sigma(i+m-2),2}, \dots, p_{\sigma(i),m}, \gamma\}.$$

The computational complexity of problem $AGV2, lp2|k \geq 2, nw|C_t$ remains open. When more than one AGV is employed, we need to consider the avoidance of conflicts. Under the NW policy, the time an AGV stays at any machine while dropping off and picking up jobs is 2ε . Therefore, in order to maintain conflict-free AGV operation using the zone control approach, there must be a time interval of at least $\theta = \delta + 2\varepsilon$ between two consecutive AGV dispatches. The Gantt chart for a sequence σ under the NW policy with two machines and two AGVs is shown in figure 3. The length of each block j of the Gantt chart is $g_j = \max\{a_{\sigma(j)}, b_{\sigma(j-1)}, \theta\}$. The idle time after block j is denoted by h_j , as in figure 3. This idle time is caused by a finished job waiting to be picked up, due to the unavailability of an AGV. In this case, the cycle time does not have a closed form expression.

Figure 3. Sequence σ under the NW policy.

We note that the Gantt chart shown in figure 3 for a two machine loop with two AGVs can be generalized to loops with $m \geq 2$ machines and $v \geq 2$ AGVs. In problem $AGVv, lpm|k \geq 2, nw|C_t$, the cycle time of an MJS in a job sequence σ can be found from the block lengths, as shown in figure 3.

4.2. Finding a fleet size

For a given MJS, the cycle time can sometimes be decreased by increasing the number of AGVs. But there is a minimum cycle time, which is a function of the job sequence and the processing times, beyond which adding more AGVs does not reduce cycle time further. Finding this minimum cycle time, the job sequence that achieves the minimum cycle time, the number of AGVs needed to achieve the minimum cycle time, and defining the tractability of these problems, are the topics discussed in this section. The decision problem is: does there exist a sequence with cycle time $C_t = C_t^*$, which can be achieved with fleet-size $v \leq D$, where D is a given constant? Note that a trivial lower bound on the cycle time problem $AGVv, lp2|k \geq 2, nw|C_t$ is

$$C_t \geq \max \left\{ n\theta, \sum_{i=1}^n a_i, \sum_{i=1}^n b_i \right\}.$$

We consider problem $AGV, lp2|k \geq 2, nw, C_t = C_t^*|v$. Note that this problem can be shown to be unary NP-hard. The proof is similar to the proof given for problem $AGV3, lp2|k \geq 2, nw|C_t$ in Hall, Sriskandarajah and Ganesharajah [17]. We now describe and test a heuristic procedure for finding an appropriate fleet size. The sequence found by the algorithm of Gilmore and Gomory [10], which minimizes $\sum_{i=1}^n \max\{a_{\sigma(i+1)}, b_{\sigma(i)}, \theta\}$, is one with the lowest possible cycle time. The Gantt chart of this sequence may be viewed as the concatenation of n consecutive blocks, each of length $\max\{a_{\sigma(i+1)}, b_{\sigma(i)}, \theta\}$, for $i = 1, \dots, n$, as in figure 3. In this sequence, if all blocks are greater than or equal to γ in length, one AGV is sufficient to obtain the minimum possible cycle time. Otherwise, the minimum cycle time with the same sequence can be achieved by dispatching an additional AGV whenever $\gamma > \max\{a_{\sigma(i+1)}, b_{\sigma(i)}\}$. Note that $\gamma \geq \theta$ in every case. But the minimum time interval at which AGVs can be dispatched, if conflicts are to be avoided, is θ . Using these ideas, an upper bound on the AGV fleet size needed to achieve the minimum cycle time, C_t^* , can be found by the following heuristic.

Heuristic FleetSize.

Step 1. Initialize $t = 1$ and $r = 1$.

Step 2. Obtain an optimal Gilmore and Gomory sequence to minimize

$$C_t = \sum_{i=1}^n \max\{a_{\sigma(i+1)}, b_{\sigma(i)}, \theta\}.$$

Step 3. Begin from block r and form a group t of consecutive blocks with total length less than γ , where the addition of the next block in the sequence increases the total length to at least γ .

Step 4. Count the number of blocks, μ_t , in group t . Set $r = r + 1$ and $t = t + 1$. If $r \leq n$, then go to step 3.

Step 5. Terminate. An upper bound on fleet size is given by $v_F = \max_t \{\mu_t\} + 1$.

In the sequence found in step 2, if the length of each block is greater than or equal to γ , one AGV is sufficient to achieve the minimum cycle time. If a block length is less than γ , then the number of AGVs required is determined by how many such consecutive blocks add up to the length of at least γ . The fleet size is the maximum value of these AGV requirements among the groups of consecutive blocks.

The main result of this subsection now follows.

Theorem 1. $\lceil n\gamma/C_t \rceil \leq v \leq v_F$.

Proof. The first bound is derived as follows. Divide the cycle length into n equal time blocks, each of duration C_t/n . Consider an AGV that is used to transport job $J_{\sigma(1)}$. This AGV can next be used to transport job $J_{\sigma(v+1)}$, where v is the smallest index such that blocks $1, \dots, v$ contain sufficient time for the AGV to complete its cycle of duration γ . Thus, v is the smallest integer satisfying $\gamma \leq vC_t/n \Rightarrow v = \lceil n\gamma/C_t \rceil$. The second inequality follows from the fact that σ is a feasible sequence, and v_F is a feasible number of AGVs when using sequence σ , which follows from the construction in Heuristic FleetSize. $\square$

We perform a computational study on Heuristic FleetSize. Processing times a_i and b_i for $i = 1, \dots, n$ are randomly generated integers in the range $1, \dots, 100$, using the built-in random number generator of the C compiler. All testing is performed on a Sun Sparc Station 20 Model 50. For each MJS, we find the smallest possible cycle time using the algorithm of Gilmore and Gomory [10]. Then we apply Heuristic FleetSize. The resulting fleet size is compared with the lower bound given by $\lceil n\gamma/C_t \rceil$. The mean upper bound (*mub*), mean lower bound (*mlb*), and mean difference ($md = mub - mlb$) are shown in table 2. Each entry in the table represents the mean over 50 randomly generated data sets. The gap between the upper and lower bounds increases slowly as the number of jobs, n , increases. For $n \geq 100$, there is some evidence that this gap

Table 2
Performance of Heuristic FleetSize.

n	Values	$\gamma = 50$	$\gamma = 100$	$\gamma = 200$	$\gamma = 300$	$\gamma = 400$
10	<i>mub</i>	2.62	3.74	5.50	7.30	8.66
	<i>mlb</i>	1.16	2.16	4.08	5.86	7.72
	<i>md</i>	1.46	1.58	1.42	1.44	0.94
50	<i>mub</i>	3.88	5.66	8.96	11.04	13.16
	<i>mlb</i>	1.70	3.18	6.14	9.10	11.86
	<i>md</i>	2.18	2.48	2.82	1.94	1.30
100	<i>mub</i>	4.18	5.86	9.40	11.82	14.94
	<i>mlb</i>	1.50	2.84	5.54	8.26	10.94
	<i>md</i>	2.68	3.02	3.86	3.56	4.00
150	<i>mub</i>	4.42	6.20	9.62	12.92	15.22
	<i>mlb</i>	1.68	3.06	5.84	8.60	11.38
	<i>md</i>	2.74	3.14	3.78	4.32	3.84
200	<i>mub</i>	5.46	8.28	13.82	17.40	22.68
	<i>mlb</i>	2.44	4.54	8.74	12.96	17.18
	<i>md</i>	3.02	3.74	5.08	4.44	5.50
250	<i>mub</i>	5.26	7.84	12.88	16.60	20.86
	<i>mlb</i>	2.26	4.26	8.24	12.24	16.22
	<i>md</i>	3.00	3.58	4.64	4.36	4.64

increases slowly as the total turnaround time of an AGV, γ , increases. Both these results are in line with our intuition, as we now discuss. First, the lower bound approximation in theorem 1 is likely to become weaker as n and γ increase. Second, the optimality provided by the Gilmore–Gomory sequence in step 2 of Heuristic FleetSize is likely to become less relevant if γ is large and thus the turnaround time is a dominant constraint. Nevertheless, the gap is small in each case, suggesting that a good approximation to the optimal fleet size is achieved here. Also, the computation times for all problems tested are very small. The upper bounds shown in table 2 provide a useful starting point for simulation studies that significantly reduces the amount of computation required there.

5. Loop decomposition

A large flowshop employing many AGVs in a single loop can be decomposed into several single AGV loops operating in tandem. We make the natural assumptions that the turnaround time of each decomposed single AGV loop is proportional to the number of machines in that loop, and that the total of the turnaround times of all the decomposed loops is equal to the turnaround time of the original large flowshop loop. For example, a five machine loop employing three AGVs can be decomposed as shown in figure 4. B_1 and B_2 denote buffers required to transfer jobs between loops.

The decomposition in figure 4 can be described as $(2, 1, 2)$, since there are two machines in the first loop, one machine in the second loop, and two machines in the third

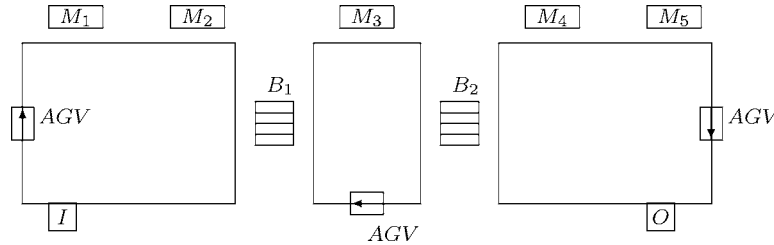


Figure 4. A decomposed system.

loop. There are five other possible decompositions in this example: $(1, 2, 2)$, $(2, 2, 1)$, $(3, 1, 1)$, $(1, 3, 1)$ and $(1, 1, 3)$. In this tandem AGV configuration, the loop with the maximum cycle time, which we refer to as the *critical loop*, defines the overall cycle time of the entire flowshop. If we find the optimal job sequence for each loop, the maximum of the optimal cycle times is the lowest possible cycle time of the entire system. But a significant practical problem may arise if different sequences are used in different loops. The buffer requirement needed to resequence jobs during transfer between the loops can be as large as n in the worst case. This conflicts with the objective of minimum buffer usage in an AGV system. In order to minimize the buffer requirement between adjacent loops, it is recommended to use the same job sequence in all the decomposed loops. Moreover, this eliminates the need to spend time resequencing the jobs in the buffers.

Kouvelis and Kim [22] determine the best sequence in which to arrange machines in a loop jobshop layout with the objective of minimizing the total material flow. They assume the existence of sufficient buffer capacity in front of each machine, and suggest decomposing a large loop into smaller loops using information contained in the from-to chart. The *concurrent engineering* philosophy [23] motivates linkages between design and operational issues. Kusiak and He [24] provide rules for the design of products so that scheduling becomes easier in a general manufacturing context. In this section, we also provide such a linkage. Our study is relevant for a manufacturing layout which is flexible in the sense that it can be reconfigured quickly to adopt the changes in MJS. This type of flexible layout is possible using *free ranging AGV systems* which do not use a physical guideway and hence are also called *virtual path AGV systems*, for which the flow path exists only in computer memory.

This section provides a detailed analysis of the advantages and disadvantages of tandem configurations relative to conventional ones. We propose analytical and computational methods for comparing the cycle times of the two AGV-served systems. We use the heuristic procedure developed by Hall, Sriskandarajah and Ganesharajah [17] for the intractable problems. This heuristic performs well, relative to lower bounds, in computational testing. We also describe conditions under which it is advantageous to decompose a large single loop into smaller tandem loops, using the objective of minimizing cycle time. All the results in this section are derived for the NW dispatching policy, where there exists a closed form expression for the cycle time in a system served by any number of AGVs under that policy, as discussed in section 4.1. In section 5.1, we consider the decomposition of a loop system served by multiple AGVs into several

single-AGV loops. In section 5.2, we consider the decomposition of a single-AGV system into smaller loops, each of which has a single, cheaper AGV.

5.1. Decomposing a loop served by multiple AGVs

In the main result of this section, theorem 2, we identify a sufficient condition under which decomposition is advantageous. We assume that the initial loop uses a job sequence σ . As discussed in the companion paper [17], for many AGV-served manufacturing configurations, finding σ is an intractable problem. In such cases, σ may be a suboptimal solution obtained by a heuristic procedure. We also assume that this loop employs a sufficient number of AGVs that there is no idle time while waiting for an AGV. That is, $h_i = 0$ for all i between blocks in schedule σ in figure 3. This sufficient number of AGVs for the given sequence σ can be found by a procedure similar to Heuristic FleetSize in section 4.2.

Theorem 2. Consider an m machine flowshop loop that uses v AGVs to minimize cycle time. Let T_σ^S (respectively, T_σ^D) denote the cycle time of a single loop (respectively, of a system of tandem loops) when the jobs are processed in sequence σ . If $\max_j \{\min_i \{p_{i,j}\}\} \geq \lceil m/v \rceil \gamma / m$, then there exists a decomposition where $T_\sigma^D \leq T_\sigma^S$.

Proof. We consider two cases.

Case 1. $v \leq m$.

In the original single loop, the cycle time is

$$T_\sigma^S = \sum_{i=1}^n \max\{p_{\sigma(i+m-1),1}, p_{\sigma(i+m-2),2}, \dots, p_{\sigma(i),m}, \theta\}.$$

Note that v is the minimum number of AGVs needed to ensure that there is no idle time while waiting for an AGV between blocks in schedule σ , and hence no γ term appears in the cycle time. This loop is now decomposed into v single AGV loops as follows. Let $z = v \lceil m/v \rceil - m$. The number of machines in each of $(v - z)$ loops is $\lceil m/v \rceil$, and the number of machines in each of the z remaining loops is $\lceil m/v \rceil - 1$. The turnaround time of the first $(v - z)$ loops is $\lceil m/v \rceil \gamma / m$, and that of the remaining loops is $(\lceil m/v \rceil - 1) \gamma / m \leq \lceil m/v \rceil \gamma / m$. The cycle time of a decomposed single AGV loop u is

$$T_{\sigma,u}^D = \sum_{i=1}^n \max\left\{p_{\sigma(i+m_u-1),n_u-m_u+1}, p_{\sigma(i+m_u-2),n_u-m_u+2}, \dots, p_{\sigma(i),n_u}, \frac{m_u \gamma}{m}\right\},$$

where m_u is the number of machines in that loop, $n_u = \sum_{k=1}^u m_k$, for $u = 1, \dots, U$, and U is the number of loops, which is equal to v in this case. The cycle time of the decomposed system as a whole is $T_\sigma^D = \max_u \{T_{\sigma,u}^D\}$. Since no AGV conflict can occur in single AGV loops, there is no θ term in the cycle time.

Since there are m machines in the original loop and $l \geq m + 2$, the turnaround time in the original loop, γ , satisfies $\gamma \geq m(\delta + 2\varepsilon) + 2\delta + 2\varepsilon$. Thus, $\theta < \gamma / m$. Further, since

$\lceil m/v \rceil \geq 1$, we can write $\theta < \lceil m/v \rceil \gamma/m$. By comparing the cycle time of the original system and the overall cycle time of the decomposed system, we can show that $T_{\sigma,u}^D \leq T_{\sigma}^S$ for each loop u , due to the condition in the theorem, the fact that $\theta < \lceil m/v \rceil \gamma/m$, and the following discussion. To compute the cycle time in the original loop, the largest of m processing times (defined as the block, g_i , as in figure 3) is selected and all such n blocks are added. But in the decomposed system, the largest of a *smaller number* $m_u < m$ of processing times is selected, n blocks are added in each loop, and the maximum cycle time over the loops is taken as the cycle time of the decomposed system. Here we have to consider two cases depending on whether the same sequence (σ^*) is adopted in each decomposed loop (case 1.1), or different sequences (σ_u^* , $u = 1, \dots, U$) are adopted in each decomposed loop (case 1.2).

Case 1.1. If we adopt a common sequence, σ^* , for all the decomposed loops, then the overall cycle time of the decomposed system is less than or equal to the original cycle time, as explained above, i.e., $T_{\sigma^*}^S \geq \max_u \{T_{\sigma^*,u}^D\}$. Let σ' be an optimal common sequence for the decomposed system, where $\sigma' \neq \sigma^*$. Then, by definition,

$$T_{\sigma^*}^S \geq \max_u \{T_{\sigma^*,u}^D\} \geq \max_u \{T_{\sigma',u}^D\}.$$

Case 1.2. If we use the optimal sequence in each loop $u = 1, \dots, U$, then the cycle time of each loop is less than or equal to the corresponding cycle time of case 1.1, i.e., $\max_u \{T_{\sigma',u}^D\} \geq \max_u \{T_{\sigma_u^*,u}^D\}$. Since the cycle time in case 1.1 is smaller than or equal to that in the original AGV system, the same result holds for case 1.2.

Case 2. $v > m$.

In this case only m AGVs are used to create m single AGV loops. The turnaround time of each loop is γ/m . Since this turnaround time is equal to $\lceil m/v \rceil \gamma/m$, the result holds. The use of an additional $v - m$ AGVs in the decomposed loops will not increase T_{σ}^D . $\square$

Corollary 1. If $\max_j \{\min_i \{p_{i,j}\}\} \geq \gamma - \gamma/m$, then any decomposition has $T_{\sigma}^D \leq T_{\sigma}^S$.

Proof. Note that, in any decomposed loop, the turnaround time is less than or equal to $\gamma - \gamma/m$. The remainder of the proof is similar to that of theorem 2. $\square$

In the following lemma, we identify a case where decomposition is disadvantageous.

Lemma 2. If $p_{i,j} \leq \gamma/v$, for $i = 1, \dots, n$ and $j = 1, \dots, m$, then any decomposition is disadvantageous.

Proof. In this case, any block length is less than or equal to γ/v in the single loop. An upper bound on the cycle time of the single loop is found by allowing idle time of $\gamma/v - g_j$ after each block j . Hence the cycle time of a single loop is at most $n\gamma/v$. The average number of machines in the tandem loops is m/v , therefore $m_u^{\max} \geq m/v$. Thus, $\gamma_u^{\max} \geq \gamma/v$. Any decomposition will have $C_t \geq n\gamma m_u^{\max}/m$, which is greater than or equal to

Table 3
Decomposition test for two machines and two AGVs.

γ	Lower bound	GA C_t	CPU sec	Loop1 C_t	Loop2 C_t	Decompose?
30	5207	5207	27	5204	4942	Yes
40	5207	5207	21	5204	4942	Yes
50	5207	5207	25	5204	4942	Yes
100	5207	5207	24	5204	4942	Yes
125	5283	5289	21	5284	5082	Yes
140	5283	5304	25	5314	5134	No
150	5207	5287	20	5356	5175	No
200	5207	5675	23	5797	5668	No
250	6250	6386	24	6500	6438	No
300	7500	7500	12	7500	7500	Yes
350	8750	8750	12	8750	8750	Yes
400	10000	10000	12	10000	10000	Yes

$n\gamma/v$, therefore the cycle time of the largest decomposed single AGV loop is no smaller than the original cycle time. It follows that any decomposition is disadvantageous. $\square$

We now perform a computational experiment on a randomly generated example problem to illustrate whether decomposing an AGV-served loop is advantageous. Consider a loop with two machines and two AGVs. When we decompose this system into two single machine, single AGV loops, the turnaround time of each decomposed loop is assumed to be $\gamma/2$. Finding the cycle time of the single loop is a unary NP-hard problem, therefore we apply the genetic algorithm of Hall, Sriskandarajah and Ganesharajah [17]. Finding the cycle time of a single machine single AGV loop is trivial, and the cycle time is $\sum_{i=1}^n \max\{a_i, \gamma/2\}$. In this example problem, we consider a job set with 50 jobs, where processing times on each machine are uniformly distributed random integer numbers between 50 and 150. For various γ values, we compare the original cycle time given by the genetic algorithm (GA C_t) with the maximum of the two single loop cycle times ($\max\{\text{Loop1}C_t, \text{Loop2}C_t\}$). We let $\theta = \gamma/10$, which appears to be a realistic assumption. Our genetic algorithm uses population size = 50, probability of crossover = 1, probability of mutation = 0.05 and number of generations = 100. The results are reported in table 3. In every case, the computation time required was less than half a minute on a Sun Sparc Station 20 Model 50.

We now discuss our computational results for the problems tested in table 3. If the value of γ is low (≤ 125), then decomposition is advantageous. Even though the reduction in cycle time is not always large, we recommend decomposition for other advantages such as freedom from conflicts, easier control, and the possibility of bidirectional operation of AGVs which could further reduce cycle time. If $\gamma \geq 300$, then decomposition does not change the cycle time. Here also, we recommend decomposition for the advantages mentioned above. For intermediate values, say $140 \leq \gamma \leq 250$, management should weigh the decrease in productivity due to decomposition against the above advantages in order to make a decision about decomposition.

5.2. Decomposing a loop served by a single AGV

Here we consider decomposing a single AGV loop into several single AGV loops, where the decomposition of the AGV system is performed so that the turnaround time, γ , of the original AGV system and the turnaround time of each decomposed single AGV loop are the same. This can be achieved by using slower, and thus cheaper, AGVs for the decomposed loops. For example, if a four machine AGV system is decomposed into two AGV loops containing two machines each, then the speeds of the AGVs that are employed in the decomposed AGV loops are assumed to be half the speed of the AGV in the original loop. In this case, if U AGVs of speed S and one AGV of speed US cost about the same, then the following theorem suggests that it is advantageous to decompose the large AGV system into U smaller loops, each of which is served by a single AGV.

Theorem 3. When the turnaround time of the original loop and that of each decomposed loop are the same under an NW policy, the decomposition of a large AGV loop system employing a single AGV into several smaller AGV loop systems each served by one AGV reduces or leaves unchanged the cycle time.

Proof. Consider the large AGV system loop shown in figure 1 and assume that only one AGV is employed in the loop. If AGV dispatching is performed according to the NW policy, the cycle time for producing one MJS is given by

$$T_{\sigma}^S = \sum_{i=1}^n \max\{p_{\sigma(i+m-1),1}, p_{\sigma(i+m-2),2}, \dots, p_{\sigma(i),m}, \gamma\}.$$

Suppose that this loop is divided into U loops each served by one AGV, where the turnaround time, γ , is the same. The number of machines in the u th loop is denoted by m_u , where $\sum_{u=1}^U m_u = m$. Let $n_u = \sum_{k=1}^u m_k$, $u = 1, \dots, U$. The cycle time of the u th decomposed loop is

$$T_{\sigma,u}^D = \sum_{i=1}^n \max\{p_{\sigma(i+m_u-1),n_u-m_u+1}, p_{\sigma(i+m_u-2),n_u-m_u+2}, \dots, p_{\sigma(i),n_u}, \gamma\}.$$

The overall cycle time of the decomposed system is equal to

$$\begin{aligned} T_{\sigma}^D &= \max_u \sum_{i=1}^n \max\{p_{\sigma(i+m_u-1),n_u-m_u+1}, p_{\sigma(i+m_u-2),n_u-m_u+2}, \dots, p_{\sigma(i),n_u}, \gamma\} \\ &= \max_u \{T_{\sigma,u}^D\}. \end{aligned}$$

By comparing the cycle time of the original AGV system, and the overall cycle time of the decomposed system, we can show by using logic similar to that of theorem 2 that $T_{\sigma,u}^D \leq T_{\sigma}^S$ for each loop u . Thus, the cycle time of the decomposed system is less than or equal to the original cycle time. Here we have to consider two cases depending

on whether the same sequence (σ^*) is adopted in each decomposed loop (case 1), or different sequences (σ_u^* , $u = 1, \dots, U$) are adopted in each decomposed loop (case 2).

Case 1. Similarly to the proof of case 1.1 of theorem 2, we can show that

$$T_{\sigma^*}^S \geq \max_u \{T_{\sigma^*,u}^D\} \geq \max_u \{T_{\sigma',u}^D\},$$

where σ' is an optimal common sequence for the entire decomposed system.

Case 2. If we use the optimal sequence, σ_u^* , in each loop $u = 1, \dots, U$, then the cycle time of each loop is less than or equal to the corresponding cycle time of case 1, i.e.,

$$\max_u \{T_{\sigma',u}^D\} \geq \max_u \{T_{\sigma_u^*,u}^D\}.$$

Since the cycle time in case 1 is smaller than or equal to that in the original AGV system, the same result holds for case 2. $\square$

Corollary 2. Given a large AGV system with m machines and one AGV of speed mS , the cycle time is reduced or remains unchanged if the loop is decomposed into m smaller loops, where each smaller loop contains one machine and employs one AGV of speed S .

Proof. Since the turnaround time of each decomposed loop is no larger than that of the original loop, the result follows immediately from theorem 3. $\square$

6. Joint sequencing

If resequencing of parts in the buffers between loops is possible, then the part sequencing problem is solved in each of the R loops, to find optimal sequences $\sigma_1, \dots, \sigma_R$ for each of these loops. The maximum cycle time T^* of the loops is the cycle time for the overall system. To accommodate differences in the optimal sequences of the loops, we may have to provide a large buffer (in the worst case, of size n) between the loops in steady state production. Alternatively, consider the case where resequencing of parts between the loops is not possible. Thus, we need to use the same sequence in each loop that minimizes the cycle time of the overall system. This is called the joint sequencing problem. In this case, the buffer requirement will typically be smaller, although not zero, as discussed in the following paragraph. If $\sigma_1 = \dots = \sigma_R$, then that sequence is optimal for the overall problem. Otherwise, we solve the overall problem R times by imposing on all the loops the sequence σ_i , $i = 1, \dots, R$, for each fixed σ_i . This procedure requires making tradeoffs in the part sequencing problem in one loop, so as not to enter another loop with a part sequence that would generate very poor performance there. The best sequence, σ^* , found this way, is a heuristic, feasible solution to the problem where resequencing is not allowed. If σ^* does not increase the maximum cycle time of the loops beyond T^* , then it is optimal. In order to avoid unnecessary buffer build up which is expensive in inventory holding cost, it is necessary to equalize the cycle time in all

loops. This can be achieved by slowing down AGVs for the loops whose cycle times are smaller than the maximum cycle time of any loop.

Even assuming that no resequencing of parts between the loops is possible, a buffer may still be needed between the loops. This is because, since the data may differ between two consecutive loops, the interval between the times at which two parts leave a loop may be different from the interval between the times at which they enter the next loop.

The joint sequencing problem (JS) is polynomially solvable in some cases, as we now discuss.

Theorem 4. Problem JS is trivial if the original loop is divided into many single machine single AGV loops.

Proof. Any sequence is optimal for a single machine single AGV loop. Therefore any sequence is optimal as a joint sequence. Note that this kind of decomposition occurs when $v = m$. $\square$

Theorem 5. Problem JS can be solved in polynomial time if the original loop is divided into single AGV loops in which one loop has two machines and every other loop has one machine.

Proof. The optimal sequence of the two machine single AGV loop can be found in polynomial time using the algorithm of Gilmore and Gomory [10]. For single machine single AGV loops, any sequence is trivially optimal. Therefore, the optimal sequence of the two-machine loop can be used in all the other loops as well. $\square$

The proof of the following result appears in appendix A.

Theorem 6. The recognition version of JS is binary NP-complete in any decomposition containing single AGV loops in which at least two loops have two or more machines and all other loops have one machine.

Theorem 7. The recognition version of JS is unary NP-complete in any decomposition containing one or more loops which have at least two machines and three AGVs.

Proof. This follows from the fact that the recognition version of problem AGV3, $lp2|k \geq 2, nw|C_i$ in unary NP-complete, as proved by Hall, Sriskandarajah and Ganesharajah [17]. $\square$

7. Concluding remarks

In this paper we present an approach to finding a minimum fleet size for an AGV-served flowshop loop. We also study the decomposition of a large AGV-served flowshop loop into smaller loops from the perspective of productivity enhancement. Also, our complexity map of joint sequencing problems motivates the development of heuristics for

intractable problems. Our study suggests several issues for further investigation. Once the decision about decomposing a multiple AGV loop into many single AGV loops is made, the possibility of operating an AGV in bidirectional mode may be explored. Although most current AGV systems employ unidirectional AGVs for easier control, the use of bidirectional mode may reduce AGV travel time, thereby leading to reduced cycle time. The bidirectional operation of an AGV in a loop is mathematically equivalent to scheduling robot-served cells [15,16,20]. Thus, valuable insights may be gained from the literature on robotic cell design and scheduling. It is useful to note that even some loops that would be less efficient if decomposed using the methodology of this paper may be worth decomposing if bidirectional operation is available. We hope that our work will motivate further research on the design and operation of automated material handling systems.

We consider the fleet size optimization problem for unit load AGVs operating in single loops. Fleet size optimization for multiple load carrying AGVs is studied by Leung, Khator and Kimbler [25], Ozden [30], and Johnson and Brandeau [19]. This alternative is appealing, since fewer AGVs are needed if they carry multiple loads. This reduces the likelihood of vehicle congestion. Analytical studies are needed to evaluate such alternatives.

Acknowledgments

This research was supported in part by the Natural Sciences and Engineering Research Council of Canada (Grant no. OGP0104900), The University of Texas at Dallas, and by the Summer Fellowship Program (Fisher College of Business, The Ohio State University). Two anonymous referees provided helpful comments on an earlier draft of this work.

Appendix A. Proof of theorem 6

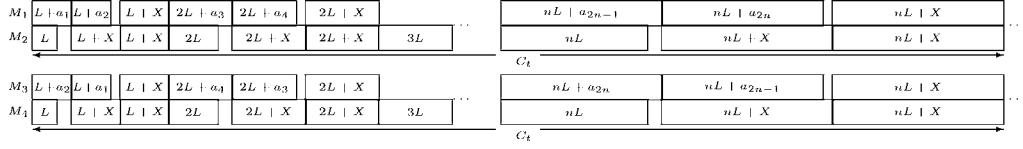
By reduction from the following binary NP-complete problem.

Even–Odd Partition [9]. Given an integer n and a set $A = \{a_1, \dots, a_{2n}\}$ of positive integers where $\sum_{i=1}^{2n} a_i = 2B$, and where $a_i < a_{i+1}$ for $1 \leq i < 2n - 1$, does there exist a partition of A into subsets A_1 and A_2 , such that $\sum_{i \in A_1} a_i = \sum_{i \in A_2} a_i = B$, and such that A_1 and A_2 each contains exactly one of $\{a_{2i-1}, a_{2i}\}$, for $1 \leq i \leq n$?

Consider the following problem instance with $3n$ jobs. We have two 2-machine loops operating in tandem. The processing times of the jobs $\{Ja_{2i-1}, Ja_{2i}, Jo_i \mid 1 \leq i \leq n\}$ are given below:

The processing times of the jobs on the first machine (M_1) are:

$$p_1(Ja_{2i-1}) = iL + a_{2i-1}, \quad p_1(Ja_{2i}) = iL + a_{2i}, \quad p_1(Jo_i) = iL + X, \quad i = 1, \dots, n.$$

Figure 5. A schedule with $C_t = C$.

The processing times of the jobs on the second machine (M_2) are:

$$\begin{aligned} p_2(Ja_{2i-1}) &= iL + X, & p_2(Ja_{2i}) &= iL + X, & i &= 1, \dots, n, \\ p_2(Jo_i) &= (i+1)L, & i &= 1, \dots, n-1, & p_2(Jo_n) &= L. \end{aligned}$$

The processing times of the jobs on the third machine (M_3) are:

$$p_3(Ja_{2i-1}) = iL + a_{2i}, \quad p_3(Ja_{2i}) = iL + a_{2i-1}, \quad p_3(Jo_i) = iL + X, \quad i = 1, \dots, n.$$

The processing times of the jobs on the fourth machine (M_4) are:

$$\begin{aligned} p_4(Ja_{2i-1}) &= iL + X, & p_4(Ja_{2i}) &= iL + X, & i &= 1, \dots, n, \\ p_4(Jo_i) &= (i+1)L, & i &= 1, \dots, n-1, & p_4(Jo_n) &= L. \end{aligned}$$

We let $X = a_{2n}$, $\gamma = L$ and $L = 3B$.

The decision problem is: does there exist a common sequence with cycle time

$$C_t \leq C = 2nX + 3n(n+1)L/2 + B,$$

where $2B = \sum_{i=1}^n a_{2i}$? We will show that there exists a common sequence with cycle time $C_t \leq 2nX + 3n(n+1)L/2 + B$ if and only if there exists a solution to even-odd partition.

($\Rightarrow$): Consider the sequence

$$\sigma = [Ja_1, Ja_2, Jo_1, \dots, Ja_{2i-1}, Ja_{2i}, Jo_i, \dots, Ja_{2n-1}, Ja_{2n}, Jo_n].$$

If there exists a solution to even-odd partition, then the sequence σ for both tandem loops (see figure 5) requires cycle time C , since $|A_1| = |A_2|$ and $\sum_{i \in A_1} a_i = \sum_{i \in A_2} a_i$.

($\Leftarrow$): We prove a series of facts about a schedule with $C_t \leq C$.

Fact 1: The idle time on M_2 (respectively, M_4) cannot exceed B .

The total processing time on M_2 (respectively, M_4) is equal to $2nX + 3n(n+1)L/2$, therefore if $C_t \leq C$ then the total idle time on M_2 (respectively, M_4) is less than or equal to B .

Fact 2: No sequence of a type other than that shown in figure 5 can achieve the cycle time $C_t \leq 2nX + 3n(n+1)L/2 + B$.

For $k = 1, \dots, n-1$, if a job with processing time μ where $kL < \mu < (k+1)L$ on M_2 (respectively, M_4) is followed by a job with processing time β where $\beta > (k+1)L$ on M_1 (respectively, M_3), then idle time of length $\beta - \mu > B$ is created on M_2 (respec-

tively, M_4). In this case, fact 1 implies that $C_t > C$. Similar arguments rule out all possible sequence types other than that shown in figure 5. The cycle time of this sequence is

$$C_t = \max \left\{ 2nX + 3n(n+1)\frac{L}{2} + \sum_{i=1}^n I_i, 2nX + 3n(n+1)\frac{L}{2} + 2B - \sum_{i=1}^n I_i \right\},$$

where I_i is the idle time on M_2 . Since $C_t = 2nX + 3n(n+1)L/2 + B$, there exists a solution to even-odd partition.

References

- [1] K.R. Baker, *Elements of Sequencing and Scheduling*, Paperback edition (Dartmouth College, Hanover, NH, 1992).
- [2] J. Blazewicz, H.A. Eiselt, G. Finke, G. Laporte and J. Weglarz, Scheduling tasks and vehicles in a flexible manufacturing system, *International Journal of Flexible Manufacturing Systems* 4 (1991) 5–16.
- [3] J. Blazewicz, R.E. Burkard, G. Finke and G.J. Woeginger, Vehicle scheduling in two-cycle flexible manufacturing systems, *Mathematical and Computer Modeling* 20 (1994) 19–31.
- [4] Y.A. Bozer and M.M. Srinivasan, Tandem configurations for AGV systems offer simplicity and flexibility, *Industrial Engineering* 21 (1989) 23–27.
- [5] Y.A. Bozer and M.M. Srinivasan, Tandem configurations for automated guided vehicle systems and the analysis of single vehicle loops, *IIE Transactions* 23 (1991) 72–82.
- [6] Y.A. Bozer and M.M. Srinivasan, Tandem AGV systems: A partitioning algorithm and performance comparison with conventional AGV systems, *European Journal of Operational Research* 63 (1992) 173–191.
- [7] H.-G. Choi, H.-J. Kwon and J. Lee, Traditional and tandem AGV system layouts: A simulation study, *Simulation* 63 (1994) 85–93.
- [8] T. Ganesharajah, N.G. Hall and C. Sriskandarajah, Design and operational issues in AGV-served manufacturing systems, *Annals of Operations Research* 76 (1998) 109–154.
- [9] M.R. Garey and D.S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness* (Freeman, New York, 1979).
- [10] P.C. Gilmore and R.E. Gomory, Sequencing a one state-variable machine: a solvable case of the traveling salesman problem, *Operations Research* 12 (1964) 665–679.
- [11] L. Gould, AGVS improves ergonomics, flexibility, *Modern Materials Handling* 45 (1990) 63.
- [12] R.L. Graham, E.L. Lawler, J.K. Lenstra and A.H.G. Rinnooy Kan, Optimization and approximation in deterministic sequencing and scheduling: a survey, *Annals of Discrete Mathematics* 5 (1979) 287–326.
- [13] V. Grasso, AGV-served assembly lines: Influence of sequencing and launch policies on system performances, *International Journal of Production Research* 28 (1990) 1385–1399.
- [14] W. Grosseschallau and A. Kusiak, An expert system for design of automated material handling systems (INSIMAS), *Material Flow* 2 (1985) 157–166.
- [15] N.G. Hall, H. Kamoun and C. Sriskandarajah, Scheduling in robotic cells: classification, two and three machine cells, *Operations Research* 45 (1997) 421–439.
- [16] N.G. Hall, H. Kamoun and C. Sriskandarajah, Scheduling in robotic cells: complexity and steady-state analysis, *European Journal of Operational Research* 109 (1998) 43–65.
- [17] N.G. Hall, C. Sriskandarajah and T. Ganesharajah, Operational decisions in AGV-served flowshop loops: scheduling, *Annals of Operations Research* 107 (2001) 161–188.

- [18] R.W. Hall, Cyclic scheduling for improvement, *International Journal of Production Research* 26 (1988) 457–472.
- [19] M.E. Johnson and M.L. Brandeau, Designing multiple-load automated guided vehicle systems for delivering material from a central depot, *Journal of Engineering for Industry* 117 (1995) 33–41.
- [20] H. Kamoun, N.G. Hall and C. Sriskandarajah, Scheduling in robotic cells: heuristics and cell design, *Operations Research* 47 (1999) 821–835.
- [21] H. Kise, T. Shioyama and T. Ibaraki, Automated two-machine flowshop scheduling: a solvable case, *IIE Transactions* 23 (1991) 10–16.
- [22] P. Kouvelis and M.W. Kim, Unidirectional loop network layout problem in automated manufacturing systems, *Operations Research* 40 (1992) 533–550.
- [23] A. Kusiak, *Concurrent Engineering: Automation, Tools, and Techniques* (Wiley, New York, 1993).
- [24] A. Kusiak and W. He, Design of components for schedulability, *European Journal of Operational Research* 76 (1994) 49–59.
- [25] L.C. Leung, S.K. Khator and D.L. Kimbler, Assignment of AGVS with different vehicle types, *Material Flow* 4 (1987) 65–72.
- [26] J.T. Lin, C.C.K. Chang and W.-C. Liu, A load-routing problem in a tandem-configuration automated guided-vehicle system, *International Journal of Production Research* 32 (1994) 411–427.
- [27] J.T. Lin and P.-K. Dgen, An algorithm for routing control of a tandem automated guided vehicle system, *International Journal of Production Research* 32 (1994) 2735–2750.
- [28] W.L. Maxwell and J.A. Muckstadt, Design of automated guided vehicle systems, *IIE Transactions* 14 (1982) 114–124.
- [29] Y. Monden, *Toyota Production System* (Institute of Industrial Engineers Press, Norcross, GA, 1983).
- [30] M. Ozden, A simulation study of multiple-load-carrying automated guided vehicles in a flexible manufacturing system, *International Journal of Production Research* 26 (1988) 1353–1366.
- [31] S. Panwalkar, Scheduling of a two-machine flowshop with travel time between machines, *Journal of the Operational Research Society* 42 (1991) 609–613.
- [32] M.L. Pinedo, *Scheduling: Theory, Algorithms and Systems* (Prentice-Hall, Englewood Cliffs, NJ, 1995).
- [33] H.I. Stern and G. Vitner, Scheduling parts in a combined production-transportation work cell, *Journal of the Operational Research Society* 41 (1990) 625–632.
- [34] H. Wang and S.A. Hafeez, Performance evaluation of tandem and conventional AGV systems using generalized stochastic Petri-nets, *International Journal of Production Research* 32 (1994) 917–932.
- [35] R.J. Wittrock, Scheduling algorithms for flexible flow lines, *IBM Journal of Research and Development* 29 (1985) 401–412.
- [36] J. Zygmunt, Guided vehicles set manufacturing in motion, *High Technology* 16 (December 1986).